정 재 훈

생년월일: 1998년 4월 30일

숭실대학교 IT대학 전자정보공학부 IT융합전공 · 2026년 8월 졸업 예정

E. deephoony@gmail.com | **T. 010-8104-2425** | github.com/deephoon

SUMMARY

AI/ML 기술보다 먼저 **"사용자가 무엇을 불편해하는가"**를 묻습니다.
추천시스템·LLM 서비스·EV 이상 진단 파이프라인을 직접 설계·구현하면서, 기술이 실제로 작동하려면 문제 정의가 먼저라는 것을 반복해서 확인해왔습니다.
조직을 운영하고 현장에서 고객을 대면해온 경험은 그 시각을 더 선명하게 만들었습니다.

AWARDS

AI 아이디어톤 최우수상 — 슈토피아 2025.02

UMC 7기 데모데이 최우수상 — 따따 2025.02

GALA 2024 우수상 — 선배챗 2025.01

공공빅데이터 최우수상 — 조달머신 2022.04

PROJECTS

조달머신 — 나라장터 추천시스템 2022

4인 팀 · 데이터 분석 · 추천시스템 설계

★ **조달청 공공빅데이터 경진대회 최우수상**

선택 편향 문제를 정의하고 정책 반영형 평점 체계 설계.
잠재요인 협업 필터링 RMSE 0.21 (73% 향상).

선배챗 — GPT 기반 진로 컨설팅 2024.11—25.01

3인 팀 · AI 서비스 기획 · 프롬프트 설계

★ **패스트캠퍼스 GALA 2024 우수상**

GPT 역질문으로 사용자 맥락을 수집해 맞춤 로드맵을 제안하는 양방향 구조 설계.
지침 20가지 프롬프트 구조화.

따따 — 위치 기반 추억 기록 앱 2024.12—25.02

크로스플랫폼 팀 · Android 개발

★ **UMC 7기 최우수상 · 플레이스토어 출시**

카카오 SDK 로그인 플로우 전체 구현. 리텐션 루프 설계.
API 명세 규칙 제안, UI 상태 4가지 사전 정의. 1,087 커밋 협업.

슈토피아 — 역학습 AI 학습 플랫폼 2025.01—02

아이디어톤 · 구현 설계 단계까지 진행

★ **숭실대 AI 아이디어톤 최우수상**

학생이 AI 펫에게 가르칠수록 AI가 성장하는 역학습 구조 제안.
게이미피케이션 제안. 기술 스택·MVP 로드맵.

EV Fault Co-Pilot 2025

졸업논문 · End-to-End 파이프라인 설계

1D-CNN Acc 0.9765. Temperature Scaling ECE 78.8% 개선.
위험도별 규칙 기반/GPT API 피드백 계층 분리.

SKILLS

AI / ML & LLM

Python, PyTorch, TensorFlow, Recommendation System, LLM Service Design, Prompt Engineering, Time-serises Modeling, Explainable AI, Model Calibration

Data Processing & Model Ops

pandas, scikit-learn, XGBoost, OpenCV, Streamlit, LangChain, docker

Product / App Engineering

Android, Kotlin, Jetpack Compose, MVVM, Hilt, REST API, UI State Design

Collaboration Tools

Notion, Slack, Discord, Git/GitHub

Creative / Design Tools

Figma, Photoshop Final Cut Pro, Logic Pro

EXPERIENCE

구름톤 유니브 숭실대 4기 (카카오×구름) 2025.03—09

전체 부대표 · Android 파트장

회칙·스터디 방식 설계. FE/BE/디자인 멀티 파트 조율, 1년 완수.

숭실대 중앙밴드동아리 SOMA 회장 2020—21

코로나 상황에서 공연 운영 기준을 처음부터 설계해 완수.

백화점 웰커밍 서비스 그리터 2021.07—현재

JH Communication

4년간 초면 고객 맥락 파악, 상황 적응형 응대.

EDUCATION

숭실대학교 전자정보공학부 IT융합전공

2017.03 — 2026.08 졸업 예정 | 융합전공: 스마트자동차

Silicon Valley AI Online Internship

2022.08 · 인공지능 응용능력 인증 2급

경험기술서

정재훈 · 숭실대학교 전자정보공학부 IT융합전공 (2026년 8월 졸업 예정)

"사용자가 무엇을 불편해하는가"에서 출발해, 기술로 해결 구조를 만들고 끝까지 실행하는 것을 지향합니다.

---

01 조달머신 — 나라장터 공공조달 추천시스템

2022.03 — 04 · 4인 팀 · 데이터 분석 · 추천시스템 설계 · 구현

★ 조달청 · 한국지능시스템학회 제1회 공공빅데이터 경진대회 최우수상

프로젝트 개요

나라장터에는 440만 명의 사용자가 있었지만 추천 기능이 없었고, 구매는 상위 노출 업체 몇 곳으로 반복해서 몰렸습니다. 표면적으로는 "추천 기능을 만들자"는 문제처럼 보였지만, 시작하기 전에 한 가지를 먼저 정리해야 했습니다.

"공공조달이라는 이 플랫폼에서, 좋은 추천이란 무엇인가."

일반 커머스라면 클릭률이나 구매 전환이 자연스러운 기준이 됩니다. 하지만 나라장터는 달랐습니다. 정책적으로 우선 고려되어야 하는 품목이 존재하고, 오랫동안 가시성이 높았던 공급사일수록 거래 이력이 쌓이는 구조였습니다. 거래 데이터가 시장 전체의 품질을 반영하는 게 아니라, 노출 이력의 누적을 반영하고 있었습니다. 많이 팔린 것을 더 많이 노출하는 방식은 기존 쏠림을 학습하고 강화하는 루프가 될 수 있었습니다.

그래서 팀이 가장 먼저 한 일은 모델을 고르는 것이 아니었습니다. 이 플랫폼에서 어떤 거래를 더 의미 있게 볼 것인지, 그 기준을 먼저 잡는 것이었습니다.

사용자 불편	핵심 설계	수치 성과
나라장터 440만 사용자의 선택 편향. 추천 기능 없이 상위 노출 업체에만 구매 집중. 양질의 중소기업 제품 발견 불가.	기존 구매빈도 기반 평점은 편향을 재생산. 의무구매(+10)·우선구매(+1) 등 정책 의도를 반영한 새로운 평점 체계 직접 설계.	잠재요인 협업 필터링 RMSE 0.21 (아이템 기반 0.78 대비 73% 향상). 10개 세부품목 교차 검증.

도메인 이해

나라장터 데이터를 처음 들여다봤을 때, 거래 이력 자체는 충분히 있었습니다. 문제는 그 이력이 무엇을 반영하고 있는가였습니다.

공공조달 플랫폼은 일반 소비재 시장과 구조가 다릅니다. 순수하게 품질 경쟁으로 거래가 이뤄지는 게 아니라, 정책적으로 우선 구매가 권장되거나 의무화된 품목들이 섞여 있습니다. 그런데 기존 거래 이력만 보면 이런 정책적 맥락이 전혀 드러나지 않습니다. 오히려 이미 많이 팔린 공급사, 이미 상위에 노출된 상품이 더 많은 기록을 갖고 있어서, 데이터 자체가 구조적으로 기울어진 운동장을 반영하고 있었습니다.

여기서 한 가지 판단이 생겼습니다. 이 데이터를 그대로 학습시키면, 추천 시스템은 기존 편향을 더 정교하게 재현하는 도구가 될 수 있다는 것이었습니다. 새로운 중소기업 제품이나 정책적으로 더 고려될 필요가 있는 품목은 거래 이력이 적기 때문에 모델이 볼수록 덜 추천하게 되는 역설이 발생할 수 있었습니다.

이 프로젝트에서 중요했던 건, 추천 기능을 구현하는 것이 아니라 공공조달이라는 맥락에서 어떤 추천이 더 공정하고 의미 있는가를 먼저 해석하는 일이었습니다.

기존 접근의 한계

가장 빠른 방법은 구매빈도를 그대로 평점처럼 사용해 협업 필터링을 돌리는 것이었습니다. 구현도 단순하고, 데이터도 이미 있었습니다.

하지만 이 방식은 근본적인 문제를 갖고 있었습니다. 구매빈도가 높을수록 높은 평점을 받는 구조에서는, 이미 많이 팔린 상품과 공급사가 계속 유리해집니다. 추천이 새로운 상품을 발견하게 돕는 도구가 아니라, 기존의 쏠림 구조를 더 정밀하게 강화하는 도구로 작동하게 됩니다.
공공조달은 "많이 팔린 것 = 더 좋은 것"으로 단순화하기 어렵습니다. 정책적으로 우선 고려되어야 하는 품목이 있고, 거래량만으로는 설명되지 않는 의미가 있습니다. 의무구매 대상 품목은 반드시 구매해야 하는 구조임에도 거래 이력이 단순히 빈도만 반영하면 다른 인기 품목과 동일하게 취급됩니다.

결국 팀이 내린 결론은 하나였습니다. 어떤 모델을 쓸지보다, 무엇을 점수로 볼 것인지를 먼저 정해야 한다는 것이었습니다. 입력 구조가 잘못된 상태에서는 모델을 아무리 정교하게 바꿔도 결과가 같은 방향으로 수렴하기 때문입니다.

설계 관점

이 프로젝트에서 가장 핵심적인 결정은 평점 체계를 직접 설계하는 것이었습니다.

기존 구매빈도 기반 평점을 그대로 쓰는 대신, 공공조달의 정책 의도를 점수 구조 안에 반영했습니다. 의무구매 대상 품목에는 +10, 우선구매 정책이 적용되는 품목에는 +1을 부여하는 방식으로 새로운 평점 체계를 만들었습니다. 숫자상으로는 단순한 가중치처럼 보일 수 있지만, 실질적으로는 이 플랫폼에서 어떤 거래를 더 의미 있게 볼 것인지에 대한 판단을 수치로 옮긴 작업이었습니다.

평점 기준이 잡히고 나서야 모델 비교가 의미를 가졌습니다. 잠재요인 협업 필터링과 아이템 기반 협업 필터링을 직접 구현해 나란히 비교했고, 단순히 한 번 실험하고 끝내지 않고 10개 세부품목 기준으로 교차 검증을 돌렸습니다. 결과는 잠재요인 방식의 RMSE 0.21, 아이템 기반의 0.78 대비 73% 향상이었습니다.

수치만큼 중요하게 생각했던 건 설명 가능성이었습니다. 왜 잠재요인 방식이 이 데이터 구조에 더 잘 맞는가를 논리적으로 설명할 수 있어야 했습니다. 구매 패턴의 잠재 벡터가 정책 가산 구조와 더 잘 정렬된다는 해석이 발표 논리의 뼈대가 됐고, 단순히 숫자가 좋다는 것이 아니라 왜 좋은지를 설명하는 발표로 이어졌습니다.

추천 알고리즘은 결국 입력 구조 위에서 작동합니다. 어떤 추천을 좋은 추천으로 볼 것인지가 먼저 정리되지 않으면, RMSE가 아무리 낮아도 그 숫자가 무엇을 의미하는지 설명하기 어렵습니다. 이 순서를 이 프로젝트에서 처음으로 직접 겪었습니다.

팀 구성과 나의 역할

4인 팀에서 저는 데이터 분석과 추천 시스템 설계를 맡았습니다. 구체적으로는 세 부분이었습니다.

가장 먼저, 그리고 가장 오래 시간을 쓴 건 평점 체계 설계였습니다. 구매빈도를 그대로 쓰면 편향이 반복된다는 판단 아래, 의무구매 대상에는 +10, 우선구매 정책 반영 품목에는 +1을 부여하는 구조를 만들었습니다. 이 가중치가 타당한지, 다른 값으로 설계했을 때 결과가 어떻게 달라지는지를 팀과 함께 반복해서 검토했습니다.

두 번째는 데이터 전처리와 실험 설계였습니다. 나라장터 데이터셋 두 개를 병합하고 사용자-품목 피벗테이블을 구성한 뒤, 잠재요인 협업 필터링과 아이템 기반 협업 필터링을 직접 구현해 비교했습니다. 10개 세부품목 기준 교차 검증까지 돌려서 결과 신뢰도를 팀 전체가 같이 확인했습니다.

세 번째는 결과 해석과 발표였습니다. 수치 차이만 제시하는 발표보다, 왜 잠재요인 방식이 이 데이터 구조에 더 잘 맞는지를 팀이 함께 납득할 수 있는 논리로 정리하는 게 목표였습니다. 구매 패턴의 잠재 벡터가 정책 가산 구조와 더 잘 정렬된다는 해석이 발표 논리의 뼈대가 됐고, 이 논리는 팀 내부 검토를 거치면서 더 단단해졌습니다.

협업과 개선

팀이 처음부터 같은 방향을 보고 있지는 않았습니다. 초반엔 알고리즘 성능 비교에 먼저 집중하자는 흐름이 있었습니다. 빠르게 모델을 돌려보고 결과를 확인하는 게 자연스러운 출발점처럼 느껴졌고, 그 방향 자체가 틀린 건 아니었습니다.

그런데 나라장터 데이터를 같이 들여다보면서 분위기가 달라졌습니다. 구매 이력이 특정 공급사로 쏠려 있는 구조를 팀 전체가 직접 확인했고, 이 상태에서 모델을 바꿔봐야 결과가 비슷한 방향으로 수렴할 거라는 걸 같이 느꼈습니다. 데이터를 보면서 생긴 공통 문제의식이었습니다.

그 이후 평점 기준을 먼저 정리하자는 방향으로 자연스럽게 합의가 됐습니다. 한 사람이 설득한 것보다, 데이터가 팀을 설득한 순간에 가까웠습니다. 그 전환이 프로젝트 전체 논리를 단단하게 만들었고, 발표 자리에서 심사 질문에 흔들리지 않고 답할 수 있었던 것도 그 합의가 바탕에 있었기 때문이었습니다.

러닝

조달머신은 제가 처음으로 모델보다 입력 정의를 먼저 보게 된 프로젝트였습니다.

그 전까지는 추천 시스템이라고 하면 알고리즘 종류와 성능 비교가 중심이라고 생각했습니다. 실제로 해보니 달랐습니다. 데이터에 어떤 의미를 부여하느냐에 따라 결과의 가치가 완전히 달라졌고, RMSE가 좋아도 그 숫자가 무엇을 최적화한 결과인지 설명하지 못하면 좋은 추천이라고 말하기 어렵다는 것을 직접 겪었습니다.

이후로는 AI/ML 프로젝트를 볼 때 바로 모델 구조로 들어가기보다, 데이터가 어떤 기준으로 만들어졌는지, 점수와 라벨이 무엇을 뜻하는지부터 보게 됐습니다. 숫자를 만드는 일과 그 숫자를 설명하는 일이 따로 떨어져 있지 않다는 감각도 이 프로젝트에서 처음으로 분명해졌습니다.

성장 포인트

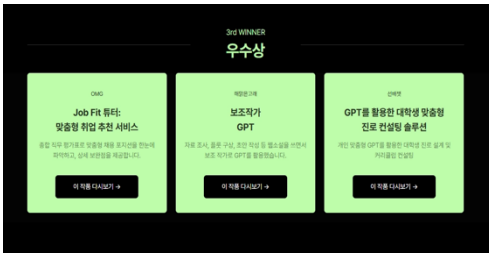
조달머신에서 생긴 이 감각은 이후 프로젝트마다 다른 형태로 반복됐습니다.

선배챗에서는 GPT가 좋은 답변을 내놓게 만드는 것이 아니라, 질문 구조 자체를 어떻게 설계할 것인가가 핵심 과제가 됐습니다. 어떤 입력이 들어가느냐가 결과의 품질을 결정한다는 원리가 그대로 적용됐습니다. EV Fault에서는 정확도 97%라는 숫자보다 그 숫자를 운전자가 믿고 행동해도 되는가, 즉 신뢰도 보정이 더 중요한 이유를 먼저 설명해야 했습니다.
조달머신에서 "RMSE 전에 평점 기준"이었던 것이, EV Fault에서는 "정확도 전에 보정"으로 바뀐 것이었습니다.
이 감각을 통해 프로젝트를 진행할 때 마다 무엇이 중요한지 다시 한번 생각하게 되는 계기가 되었습니다.

02 선배챗 — GPT 기반 대학생 맞춤형 진로 컨설팅

2024.11 — 2025.01 · 3인 팀 (기획 · 프롬프트 · UX) · AI 서비스 기획

★ 패스트캠퍼스 GALA 2024 우수상 — ChatGPT 활용 사례 부문



프로젝트 개요

진로 정보는 이미 넘쳐났습니다. 검색 한 번이면 직무 소개, 취업 로드맵, 선배 인터뷰가 쏟아졌고, GPT에게 물어도 그럴듯한 답변이 바로 나왔습니다. 그런데도 "뭘 해야 할지 모르겠다"는 말은 줄어들지 않았습니다.

선배챗은 그 간극에서 출발했습니다. 정보가 없어서가 아니라, 그 정보가 지금의 사용자와 맞닿아 있지 않기 때문에 진로 고민이 해결되지 않는다는 판단이었습니다. 자유전공제 확대로 73개 대학, 3만 8천 명의 학생이 전공 선택을 유예한 채 입학하는 시대에, 진로 고민의 출발점이 제각각인 사람들에게 동일한 정보를 나열하는 방식은 점점 더 한계가 분명해졌습니다.

그래서 선배챗이 풀려고 한 문제는 "GPT가 더 좋은 답을 하게 만드는 것"이 아니었습니다. 질문 자체가 모호한 상태의 사용자를 어떻게 구조화해서, 실행 가능한 방향으로 연결할 것인가였습니다. 단순한 답변형 챗봇이 아니라, AI가 먼저 묻고 사용자의 상태를 정리한 뒤 로드맵으로 연결하는 서비스 흐름을 설계하는 프로젝트였습니다.

사용자 불편	핵심 전환	설계 결과
기존 진로 서비스는 정보를 일방적으로 나열. 자유전공 확대(73개교, 3.8만 명)로 진로 혼란 심화. 사용자 맥락 반영 불가.	"AI가 먼저 묻는다"는 구조 전환. GPT가 역질문으로 전공·MBTI·흥미를 수집한 뒤, 맞춤형 3개월 단위 로드맵 제안.	세부 지침 20가지 프롬프트 구조화. 대화 스타터 4개 설계. 3개 유즈케이스 검증.

도메인 이해

진로 탐색이 어려운 이유는 정보가 부족해서가 아닙니다. "진로가 고민된다"는 말 안에 담긴 상태는 사람마다 전혀 다릅니다.

흥미 자체가 아직 형성되지 않은 사람이 있고, 하고 싶은 건 있지만 어디서부터 시작해야 할지 모르는 사람이 있고, 방향은 어느 정도 있는데 그걸 어떤 직무나 산업과 연결해야 할지 막혀 있는 사람도 있습니다. 이 세 가지는 겉으로 비슷해 보이지만, 실제로 필요한 개입의 방향이 완전히 다릅니다. 첫 번째 유형에게 직무 로드맵을 주는 건 의미가 없고, 세 번째 유형에게 흥미 탐색 질문을 던지는 건 오히려 답답함을 키웁니다.

그런데 기존 서비스는 이 차이를 충분히 구분하지 않았습니다. 사용자가 고민을 입력하면 비교적 평평한 답변이 나왔습니다. 틀린 정보는 아니지만, 이 사람의 현재 상태를 실제로 건드리지 못하는 답변이었습니다. 문제는 모델의 성능이 아니었습니다. 사용자가 어떤 상태에 있는지 파악하기 전에 답부터 시작하는 구조 자체였습니다.

기존 접근의 한계

초기에 별도 지침 없이 GPT를 직접 돌려봤습니다. 사용자가 진로 고민을 자유롭게 입력하면 AI가 바로 답하는 방식이었습니다.

결과는 예상대로였습니다. "관심 분야를 더 탐색해보세요", "자신의 강점을 찾아보세요", "관련 경험을 쌓아보세요" — 틀린 말은 아니지만 실제 행동으로 이어지기에는 너무 일반론적이었습니다. 같은 말을 들어도 누군가는 움직이고 누군가는 여전히 막혀 있는 이유가 여기 있었습니다. 답변이 사용자의 현재 상태와 연결되지 않으면, 아무리 정확한 정보도 그냥 스쳐 지나갑니다.

문제를 모델의 능력 부족으로 보지 않기로 했습니다. LLM은 맥락이 충분히 쌓일수록 훨씬 구체적인 결과를 냅니다. 기존 방식은 그 강점을 거의 활용하지 못하고 있었습니다. 맥락 수집 없이 답부터 시작하는 구조, 그게 실제 문제였습니다.

설계 관점

구조를 바꾸기로 했을 때 가장 먼저 정한 건 하나였습니다. AI가 먼저 답하지 않고 먼저 물어야 한다는 것이었습니다.

사용자가 입력을 하면 GPT가 역질문을 던집니다. 전공, MBTI, 흥미 같은 기본 정보뿐 아니라, 과거에 몰입했던 경험, 어떤 방식의 일을 좋아하는지, 지금 막히는 이유가 정보 부족인지 실행 부담인지까지 파악한 뒤에야 제안이 나오는 흐름으로 설계했습니다. 질문이 단순 정보 수집에 그치면 의미가 없었습니다. 사용자 스스로도 정리하지 못했던 상태를 드러내게 만드는 질문이어야 했습니다.

출력 구조도 바꿨습니다. "당신에게 맞는 직무는 A, B, C입니다"로 끝나는 리스트 형태가 아니라, 앞으로 3개월 동안 무엇을 해볼 수 있는지까지 담긴 로드맵 형태로 나오도록 했습니다. 정보를 주는 것이 아니라 다음 행동을 설계해주는 것, 그게 진로 맥락에서 실제로 쓸 수 있는 출력이라고 판단했기 때문입니다.

이 프로젝트는 프롬프트를 정교하게 다듬는 작업이라기보다, 질문 흐름과 답변 구조 전체를 하나의 제품처럼 설계하는 일에 가까웠습니다.

팀 구성과 나의 역할

3인 팀에서 저는 AI 서비스 기획과 프롬프트 구조 설계를 맡았고, 다른 팀원들은 UX 리서치와 공모전 문서화를 담당했습니다. 역할이 나뉘어 있었지만 서로 영향을 주고받는 방식으로 연결됐습니다.

직접 만든 건 세부 지침 20가지였습니다. 역할 설정, 초기 정보 수집 프로토콜, 흥미가 모호한 사용자를 위한 추가 질문 로직, 3개월 단위 실행 계획 생성 방식, 관련 기업·자격증·플랫폼 정보 연결까지 포함됐습니다.

출력 형태를 바꾼 건 팀원 피드백에서 나왔습니다. 처음엔 결과가 항목 나열 형식으로 나왔는데, UX 리서치를 맡은 팀원이 사용자 입장에서 이 형태로는 다음에 뭘 해야 할지 여전히 모호하다는 얘기를 꺼냈습니다. 리스트보다 앞으로 3개월 동안 무엇을 해볼 수 있는지가 보이는 로드맵 형태가 맞다는 방향으로 함께 정리됐고, 그 이후 출력 구조를 전면 수정했습니다. 혼자 테스트할 때는 잘 안 보이던 문제를 팀 시각이 잡아준 경우였습니다.

대화 스타터 4개를 구성하는 과정도 비슷했습니다. 진입 질문이 너무 열려 있으면 오히려 어디서 시작해야 할지 막힌다는 피드백을 반영해서, "나에게 맞는 진로 설계", "효과적인 공부방법"처럼 사용자가 부담 없이 고를 수 있는 구체적인 입구로 다시 설계했습니다.

협업과 개선

초기 방식을 버리기로 한 것도 팀 전체가 같이 결과를 보면서 내린 결론이었습니다. GPT가 바로 답하는 자유 응답형 구조로 출발했는데, 결과를 같이 놓고 보면서 "이 정도면 이미 있는 서비스와 다를 게 없다"는 얘기가 팀 안에서 나왔습니다. 한 사람이 먼저 방향을 제시했다기보다, 다 같이 출력을 보다가 동시에 같은 문제를 느낀 순간이었습니다.

그 이후 무엇을 먼저 물어야 하는지로 논의 방향이 바뀌었고, 각자 맡은 영역에서 같은 문제를 다른 각도로 확인해줬습니다. UX 리서치 결과가 프롬프트 구조와 출력 형태 수정에 영향을 줬고, 프롬프트가 바뀌면 다시 리서치 관점에서 검토하는 방식으로 작업이 맞물렸습니다. 역할이 분리돼 있었지만 결과물은 계속 한 방향으로 수렴했습니다.

러닝

선배챗 이후로 LLM을 보는 방식이 바뀌었습니다.

그 전까지는 좋은 모델이 좋은 답을 낸다는 감각이 강했습니다. 이 프로젝트를 하면서 달라진 건, 모델이 아니라 그 앞에 놓인 구조를 먼저 보게 됐다는 점입니다. 같은 모델이라도 사용자 상태를 먼저 정리하는 흐름을 거치면 출력이 완전히 달라졌습니다. 답변의 질은 결국 그 전에 어떤 질문이 몇 번 오갔느냐에서 결정됐습니다.

진로 도메인이 특히 그 감각을 선명하게 만든 이유는, 여기서는 정보 자체보다 사용자가 그 정보를 받아들일 수 있는 상태인지가 훨씬 중요하기 때문입니다. 맥락이 정리되지 않은 사람에게 아무리 정확한 직무 정보를 줘도 행동으로 이어지지 않습니다. 반대로 맥락이 먼저 정리된 사람에게는 평범한 조언도 실제 행동의 출발점이 될 수 있습니다. 이걸 직접 확인하고 나서부터는, 서비스를 설계할 때 출력보다 입력 구조를 먼저 보는 습관이 생겼습니다.

성장 포인트

선배챗 프로젝트를 통한 성장은 다음과 같은 흐름으로 이어졌습니다 .

따따에서는 "사용자가 왜 앱에 계속 돌아오는가"를 설계하는 문제로 이어졌습니다. 기능이 많아서 리텐션이 생기는 게 아니라, 사용자의 어떤 행동이 다음 방문의 이유가 되는지를 흐름으로 설계해야 했습니다. 슈토피아에서는 학습자의 설명 행동이 시스템 상태를 바꾸는 구조를 설계했는데, 이것도 결국 사용자의 현재 상태를 먼저 파악하고 그에 맞는 개입을 설계한다는 같은 원리였습니다. EV Fault에서도 운전자에게 무엇을 먼저 보여주고 어떤 층위에서 어떻게 설명할지를 나누는 사고로 연결됐습니다.

선배챗은 GPT 활용 프로젝트였지만, 실제로 배운 건 LLM의 기술적 특성이 아니었습니다. 사용자 상태를 먼저 구조화하는 것이 서비스의 출력 품질을 어떻게 바꾸는지, 그 원리를 직접 손으로 확인한 프로젝트였습니다..

03 따따 (CCI·CCI·) — 위치 기반 개인 추억 기록 앱

2024.12 — 2025.02 · 8 팀

(PM 1 + 디자이너 1 + Android 4 + Backend 4) · Android 개발

★ UMC 7기 전국 데모데이 최우수상 · 구글 플레이스토어 실 출시s

프로젝트 개요

기록 앱을 오래 쓰는 사람이 드문 이유가 뭘까, 팀이 처음 모였을 때 이 얘기를 꽤 오래 했습니다.

위치 저장, 메모, 사진 첨부. 기능만 보면 어렵지 않습니다. 그런데 그게 다면 사람들은 한두 번 써보고 앱을 닫습니다. SNS는 타인의 시선이 부담이고, 일기 앱은 써야 한다는 강박이 생기고, 지도 앱은 정보는 남아도 감정이 쌓이는 느낌이 없습니다. 결국 기록을 시작하게 만드는 것보다, 계속 남기게 만드는 게 훨씬 어려운 문제였습니다.

따따는 그 지점에서 출발했습니다. 공유 없이 나만의 추억을 위치 기반으로 남길 수 있는 앱인데, 팀이 집중한 건 기능 자체보다 기록 행동이 반복될 수 있는 구조를 어떻게 만들 것인가였습니다. 저는 Android 개발을 맡았고, 화면 구현부터 인증 흐름, 상태 관리, 팀 개발 기준 정립까지 앱이 실제로 굴러가게 만드는 전반을 담당했습니다.

사용자 불편	사용자 중심 설계	팀 협업
SNS의 타인 시선, 일기 앱의 강박, 지도 앱의 마찰. 공유 없이 나만의 추억을 기록하고 싶은 니즈가 충족되지 않음.	"왜 기록을 시작하고도 금방 그만둘까?"에서 출발. 콘텐츠를 쌓을수록 캐릭터가 성장하는 리텐션 루프 설계 제안.	API 명세 규칙 직접 제안. UI 상태 4가지 사전 합의. Gitflow 1,087 커밋 협업.

도메인 이해

개인 기록 서비스에서 진짜 문제는 기능이 아닙니다.

기록을 남기는 행동은 생각보다 금방 멈춥니다. 특별히 귀찮아서가 아니라, 남겼을 때 뭔가 쌓이는 느낌이 없으면 다시 열 이유가 생기지 않습니다. 앱을 깔고 며칠 써보다 그만두는 패턴은 거의 대부분 여기서 시작됩니다. 기록이 어딘가 축적되고 있다는 감각, 내가 남긴 것들이 눈에 보이는 무언가로 이어지고 있다는 느낌이 있어야 사람들은 계속 돌아옵니다.

이 도메인에서 기능을 더 넣는 방향은 그래서 답이 아니었습니다. 오히려 기록 한 번이 다음 기록의 이유가 되는 흐름을 앱 안에 만드는 것, 그게 핵심이었습니다.

기존 접근의 한계

CRUD가 돌아가면 앱은 완성된 것처럼 보입니다. 실제로 저도 처음엔 그렇게 생각했습니다.

그런데 개발을 진행하면서 두 가지 문제가 동시에 나왔습니다. 하나는 사용자 경험 쪽이었습니다. 기능이 붙었는데 왜 계속 쓰고 싶은 앱이 안 되는가. 위치 저장과 메모 기능만으로는 "한번 써보고 끝나는 앱"을 벗어나기 어려웠습니다. 다른 하나는 개발 구조 쪽이었습니다. Android 4명, Backend 4명이 동시에 붙어서 작업하다 보니 API 응답 구조가 조금씩 다르게 해석되고, 화면마다 상태 처리가 제각각으로 흘렀습니다. 누가 틀렸다기보다, 공통 기준이 없는 채로 병렬 개발이 돌아간 게 문제였습니다.

기능 부족과 기준 부재, 두 가지가 동시에 해결되지 않으면 앱이 커질수록 경험이 흔들린다는 걸 이 시점에서 분명히 느꼈습니다.

설계 관점

두 가지를 먼저 잡고 들어갔습니다.
첫 번째는 리텐션 루프였습니다. 기록을 남길수록 캐릭터가 성장하고 보상이 생기는 구조, 콘텐츠가 쌓일수록 앱을 다시 열 이유가 만들어지는 흐름을 설계했습니다. 기록 자체가 목적이 아니라, 기록이 다음 기록을 부르는 구조를 만드는 게 목표였습니다.

두 번째는 공통 기준이었습니다. 인증 흐름, 세션 관리, API 명세, 예외 처리, 빈 상태와 로딩 상태까지 팀이 같은 그림을 보고 있어야 화면이 늘어나도 경험이 흔들리지 않습니다. 구현을 빠르게 시작하는 것보다, 이 기준을 먼저 맞추는 데 시간을 쓰는 게 결국 더 빠른 길이라고 판단했습니다.

팀 구성과 나의 역할

PM 1명, 디자이너 1명, Android 4명, Backend 4명. 8인 팀이 6개월 동안 1,087 커밋을 올렸습니다. 규모가 커질수록 각자 잘 만든 코드들이 오히려 서로 발목을 잡는 상황이 생깁니다. Android 파트 안에서도, Backend와의 접점에서도 그 문제가 실제로 반복됐습니다.

카카오 SDK 기반 로그인 플로우를 맡아 인증, 토큰 발급, 세션 유지까지 한 흐름으로 연결했습니다. 이 과정에서 Backend 팀과 토큰 만료 처리 방식을 맞추는 작업이 병행됐습니다. 처음에는 각자 다르게 처리하고 있었고 그게 세션 불안정으로 이어졌는데, 같이 케이스를 정리하면서 기준을 맞추니 이슈가 눈에 띄게 줄었습니다.

인증 상태 관리 구조도 다시 잡았습니다. 초기 구현에서 토큰 만료 시 예외가 반복적으로 터졌는데, Android 내부 구조 문제이기도 했지만 Backend 응답 타이밍과 맞지 않는 부분도 있었습니다. ViewModel 기반 단방향 StateFlow 구조로 바꾸면서 동시에 Backend 팀과 재발급 흐름을 재합의했고, 그 이후 세션 관련 이슈가 거의 사라졌습니다.

캐릭터 커스터마이징과 아이템 구매 구조는 PM, 디자이너와 긴밀하게 맞춰가며 만든 부분이었습니다. 어떤 행동이 어떤 보상으로 이어지는지 기획 단에서 먼저 합의가 됐고, 그 구조를 Android에서 구현했습니다.

협업과 개선

API 명세 충돌은 생각보다 컸습니다. 각자 맡은 화면을 만들면서 Backend 응답 구조와 Android 쪽 기대값이 계속 어긋났고, 처음엔 발생할 때마다 양쪽에서 수정하는 방식으로 넘겼습니다. 그런데 화면이 늘어날수록 같은 종류의 충돌이 반복됐고, 어느 시점에 Backend 팀에서 먼저 명세를 다시 맞추자고 꺼냈습니다. 쌓인 이슈를 보면서 Backend 쪽에서도 패치 방식으로는 한계가 있다는 걸 느끼고 있었던 것 같습니다.

그 이후 같이 앉아서 명세를 처음부터 다시 정리했습니다. API 명세 문서 작성 규칙을 함께 정했고, UI 상태를 Loading, Empty, Error, Success 네 가지로 먼저 합의하는 방식도 이때 같이 만들었습니다. 이후 새로운 화면이 추가될 때마다 상태 처리를 따로 논의하지 않아도 됐고, 후반 개발 속도가 전반에 비해 확실히 달라졌습니다.

멀티모듈 아키텍처는 Android 파트 내부에서 먼저 논의했습니다. 초반엔 구조를 나누는 게 오히려 복잡하다는 의견도 있었습니다. 기능이 겹치면서 서로 코드를 건드리는 일이 생기기 시작했고, 그 시점에 분리 필요성에 공감대가 자연스럽게 생겼습니다. 파트 안에서 역할을 나눠 모듈 구조를 설계했고, 이후 작업이 훨씬 독립적으로 돌아갔습니다.

러닝

따따에서 배운 게 두 갈래로 정리됩니다.

하나는 제품 구조입니다. 기록 앱에서 기록 기능 자체는 사실 어렵지 않습니다. 어려운 건 사용자가 내일도 앱을 열게 만드는 것입니다. 기록 한 번이 눈에 보이는 변화로 이어지고, 그게 다음 기록의 이유가 되는 흐름, 이걸 서비스 안에 구조로 만드는 게 훨씬 본질적인 문제였습니다. 이후에 제품을 볼 때 "이 행동이 반복되게 설계되어 있는가"를 먼저 보게 된 게 이 프로젝트에서 생긴 습관입니다.

다른 하나는 개발 구조입니다. 혼자 잘 짠 코드보다, 여러 명이 같은 기준으로 움직이는 코드가 실제로 더 탄탄합니다. 상태 정의, API 명세, 예외 처리 기준처럼 눈에 잘 띄지 않는 부분이 먼저 정리되어 있어야 팀 전체 작업이 흔들리지 않는다는 걸, 이 프로젝트에서 직접 겪으면서 이해했습니다.

성장 포인트

따따에서 생긴 두 가지 러닝 포인트는 이후에도 발전할 수 있는 밑거름이 되었습니다.

사용자 행동을 반복하게 만드는 구조에 대한 감각은, 슈토피아에서 학습자의 설명 행동이 시스템 상태를 바꾸는 루프를 설계하는 방향으로 연결됐습니다. EV Fault에서도 운전자 피드백이 단발성 알림이 아니라 상태 변화와 이어지는 구조로 설계한 것이 같은 맥락이었습니다.

팀 개발 기준에 대한 감각은 더 직접적으로 이어졌습니다. 이후 구름톤 유니브에서 Android 파트장을 맡았을 때, 스프린트 시작 전에 상태 정의와 API 명세를 먼저 맞추는 방식으로 운영한 게 따따에서 직접 겪은 경험에서 나온 것이었습니다.

따따는 Android 개발 프로젝트이면서, 제품이 실제로 굴러가려면 기능 구현과 협업 구조가 같이 설계되어야 한다는 걸 몸으로 익힌 시간이었습니다.

04 슈토피아 (SSU:TOPIA) — 역학습 기반 AI 학습 플랫폼

2025.01 — 02 · 3인 팀 · AI 서비스 기획 · 아이디어톤
(구현 설계 단계, 실제 코드 구현 미진행)

★ 제1회 숭실 재학생 학습법 경진대회 AI 아이디어톤 최우수상

프로젝트 개요

AI 학습 서비스는 다 비슷한 방향으로 달립니다. 더 정확하게, 더 친절하게, 더 빠르게 설명해주는 쪽으로. 근데 저는 거기서 좀 다른 생각이 들었습니다. 설명을 잘해주는 AI가 아무리 많아진다고 해서, 학생이 진짜로 이해하게 되는 건 아닐 수 있겠다 싶었습니다.

파인만이 말한 게 딱 그겁니다. 설명할 수 있으면 이해한 거고, 막히는 지점이 있으면 거기가 모르는 곳입니다. 듣는 것보다 말하는 순간에 훨씬 많은 게 드러납니다. 그럼 반대로 가면 어떨까 싶었습니다. 학생이 AI한테 배우는 게 아니라,

학생이 AI를 가르칠수록 AI가 성장하는 구조.

슈토피아는 그 역발상에서 나왔습니다. AI가 더 잘 가르치는 서비스가 아니라, 학생의 설명 행위 자체가 학습 경험이 되고 시스템 안에 쌓이는 구조를 기획했습니다. 아이디어톤 프로젝트라 실제 코드 구현은 없었지만, 팀과 함께 기술 스택부터 MVP 로드맵까지 실제로 만들 수 있는 구조로 정리하는 데 집중했습니다.

사용자 불편	핵심 역발상	설계 범위
기존 AI 학습 도구는 모두 AI→학생 단방향. 학생은 수동적 수용자에 머무르고, 반복적 문제풀이는 학습 동기 유지 불가.	"학생이 AI를 가르친다." 파인만 학습법 — 설명할 수 있으면 이해한 것. 가르치는 행위 자체가 학습이 되는 역학습 구조.	아이디어톤으로 기술 스택·시스템 구조·MVP 로드맵까지 설계. 실제 코드 구현은 미진행. LLM + Agent + RL + LMS 연동 관점의 구조 제안

도메인 이해

학습 서비스가 오래 살아남으려면 뭐가 필요한지를 먼저 생각했습니다.

좋은 설명만으로는 부족합니다. 학생이 계속 돌아오려면 자기가 남긴 흔적이 쌓이고, 그 흔적이 어떤 변화를 만들고 있다는 감각이 있어야 합니다. 근데 기존 AI 학습 도구는 거의 다 단방향입니다. 질문하면 답이 나오고, 사용자는 계속 받는 쪽에 머뭅니다. 편하긴 한데, 오래 쓰게 만드는 구조가 아닙니다.

학습 도메인에서 핵심은 정보의 양이 아니라 개입의 강도입니다. 학습자가 얼마나 능동적으로 설명하고, 틀리고, 고쳐나가느냐가 실제 이해의 깊이를 결정합니다. 그 과정에서 시스템이 반응하고 변화한다면, 서비스는 일회성 도구가 아니라 관계가 쌓이는 무언가가 될 수 있습니다.

설계 관점

구조의 핵심은 역학습, Reverse-Teaching이었습니다.
학생이 AI 펫에게 개념을 설명하면, 그 설명 로그가 시스템 안에 남습니다. 로그가 쌓일수록 AI는 더 나은 응답을 하고, 학생 눈에는 자기가 가르친 AI가 실제로 성장하는 게 보입니다. 역할 놀이처럼 보일 수 있는데, 설계 의도는 달랐습니다. 사용자의 설명 행동이 단순 인터랙션으로 소비되는 게 아니라, 다음 경험을 바꾸는 데이터로 남는 구조를 만드는 게 목표였습니다.

기술 구조는 이 흐름을 따라 역할을 나눴습니다. 자연어 교정과 상호작용은 GPT-4, 대화 맥락을 끊기지 않게 이어주는 건 LangChain과 Vector DB, 학생 수준에 따른 난이도 조절은 PPO 기반 RL, 성장 경험의 시각화는 Unity 2D로 잡았습니다. 기술 이름을 붙이는 것보다, 이 구조 안에서 각 기술이 왜 필요한지를 설명하는 데 더 신경 썼습니다. 유행하는 스택을 채워 넣은 게 아니라, 역학습 루프가 실제로 돌아가려면 어떤 기술이 어느 자리에 있어야 하는지를 먼저 생각하고 거기서 골랐습니다.

MVP 로드맵도 단계를 나눠 정리했습니다. 처음 3~6개월은 AI 챗봇, 기본 퀘스트, 교정 로그 수집에 집중하고, 6~12개월에 AI 펫 장기 학습과 협업 기능을 붙이고, 이후에 LMS 통합과 AR/VR 확장을 고려하는 순서였습니다. 처음부터 다 넣으면 어디서 시작해야 할지 흐려집니다. 지금 당장 검증해야 할 게 무엇인지를 먼저 정해야 구조가 단단해집니다.

팀 구성과 나의 역할

역학습 구조 제안, 기술 아키텍처 설계, 게이미피케이션 설계, MVP 로드맵 구성을 맡았습니다. 아이디어톤 특성상 역할 경계가 유동적이었고, 팀 전체가 아이디어를 함께 발전시키는 방식으로 진행됐습니다.

역학습 구조는 팀 논의 안에서 점점 구체화됐습니다. 초기엔 설정 수준의 아이디어였는데, 팀원들이 "그럼 설명 로그는 실제로 어떻게 쌓이는 거야?", "AI가 성장한다는 게 사용자 눈에는 어떻게 보여야 해?"라는 질문을 계속 던졌고, 그 질문들이 구조를 구체적으로 끌어내는 역할을 했습니다. 제가 초안을 잡으면 팀이 구멍을 찾아줬고, 그 과정을 반복하면서 설계가 단단해졌습니다.

기술 아키텍처도 처음엔 방향이 여럿이었습니다. 팀원 중에 기능을 더 넣고 싶어하는 쪽이 있었고, 자연스럽게 스택도 늘어나는 방향으로 흘렀습니다. 이때 각 기술이 이 구조에서 실제로 어떤 역할을 하는지 설명할 수 있어야 한다는 기준을 팀이 같이 잡으면서, GPT-4, LangChain, PPO, Unity 2D로 역할을 나누는 방향으로 수렴했습니다.

MVP 로드맵은 제가 초안을 잡고 팀 피드백을 받아 수정했습니다. 단계를 어떻게 나눌지, 첫 번째 검증 지점을 어디에 둘지를 팀이 같이 고민했고, 그 과정에서 로드맵이 훨씬 현실적인 모양으로 다듬어졌습니다.

협업과 개선

기능을 계속 붙이고 싶어하는 흐름은 팀 전체에서 나왔고, 저도 그 안에 있었습니다. 재밌는 아이디어가 나올수록 더 넣고 싶었고, 문서가 풍성해지는 만큼 발표가 좋아 보이는 느낌도 있었습니다. 팀원 중에 기능 확장에 적극적인 사람이 있었는데, 그 에너지 덕분에 아이디어 자체는 훨씬 다양해졌습니다.

문제는 그럴수록 "이걸 어디서부터 실제로 만들 수 있지?"가 흐려진다는 거였습니다. 어느 순간 기능 목록은 늘어나는데 구현 순서가 안 잡히는 상태가 됐고, 팀 안에서도 비슷한 답답함을 느끼는 분위기가 있었습니다.

그 시점에 핵심과 확장을 나누자는 얘기를 꺼냈습니다. 기능 확장에 적극적이었던 팀원도 막상 구현 가능성을 따지면 같은 한계를 느끼고 있었고, "우리도 그게 걱정이었다"는 반응이 나오면서 방향이 빠르게 정리됐습니다. 이후 논의가 기능 나열에서 벗어나 사용자 행동이 어떤 시스템 변화를 만드는지, MVP 수준에서 어디까지 확인할 수 있는지로 이동했습니다. 방향을 바꾼 게 아니라, 팀이 이미 느끼고 있던 걸 같이 꺼낸 것에 가까웠습니다.

러닝

이 프로젝트에서 가장 선명해진 건, AI Agent형 서비스는 좋은 모델 하나로 살아나지 않는다는 것이었습니다.
사용자의 행동이 시스템 안에 남아야 합니다. 그 행동 때문에 다음 경험이 달라진다는 게 느껴져야 합니다. 그게 없으면 서비스는 결국 한 번 쓰고 끝나는 도구입니다. 슈토피아에서 설계한 역학습 루프가 정확히 그 문제를 풀려는 구조였고, 실제 구현까지는 못 갔지만 그 원리를 손으로 짚어보는 경험이었습니다.

하나 더 남은 건, 구현하지 않은 부분을 무리하게 포장하지 않는 태도입니다. 아이디어가 크다고 해서 다 된 것처럼 쓰면 오히려 신뢰가 떨어집니다. 지금 검증할 수 있는 부분과 나중으로 미룰 부분을 솔직하게 나누는 것, 그게 오히려 이 프로젝트를 더 단단하게 만들었습니다.

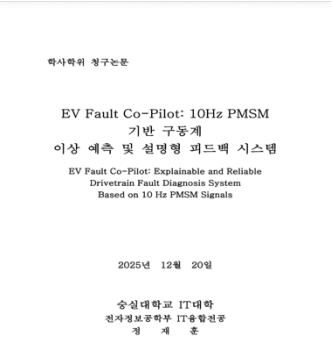
성장 포인트

슈토피아는 앞 프로젝트들에서 잡은 감각이 한 번에 모인 시간이었습니다. 조달머신에서 시작한 기준 설계, 선배챗에서 다듬은 질문 흐름과 맥락 수집, 따따에서 만진 사용자 행동 반복 구조가 여기서 같이 들어왔습니다.

EV Fault로 넘어가면서 이 감각이 또 다른 형태로 이어졌습니다. 생성형 AI를 어디까지 판단 계층에 둘 수 있는지, 어디서부터는 설명만 맡겨야 하는지를 나누는 사고가 슈토피아에서 역학습 루프를 설계하면서 생긴 시각과 연결됐습니다.
AI를 답변 엔진으로 보는 것과, 사용자 행동과 시스템 상태를 함께 설계하는 제품 요소로 보는 것은 꽤 다른 출발점입니다. 그 차이를 이 프로젝트에서 처음으로 직접 손으로 만져봤습니다.

05 EV Fault Co-Pilot — PMSM 구동계 이상 진단 시스템

2025 · 졸업논문 · 지도교수 권민혜 · 단독 수행 (파이프라인 전체 직접 설계·구현)


학사학위 청구논문

EV Fault Co-Pilot: 10Hz PMSM
기반 구동계
이상 예측 및 설명형 피드백 시스템

EV Fault Co-Pilot: Explainable and Reliable
Drivetrain Fault Diagnosis System
Based on 10 Hz PMSM Signals

2025년 12월 20일

숭실대학교 IT대학
전자정보공학부 IT융합전공
정 재 훈

프로젝트 개요

정확도 97.65%. 숫자만 보면 충분해 보입니다.

근데 이 프로젝트를 하는 내내 머릿속에서 떠나지 않은 질문이 있었습니다.
“이 결과를 운전자가 실제로 믿고 행동해도 되는가.”

전기차 구동계 센서 데이터로 이상 징후를 분류하는 시스템인데, 진단 결과가 아무리 정확해도 그 확률이 과신되거나, 설명이 모호하거나, 생성형 AI가 애매한 위치에서 판단에 끼어들면 시스템 전체 신뢰가 무너질 수 있습니다. 차량 안전 시스템에서 그건 단순한 서비스 품질 문제가 아닙니다.

그래서 이 프로젝트는 분류 모델을 잘 만드는 것에서 시작해서, 그 결과를 어떤 계층을 거쳐 어떤 방식으로 전달해야 사람이 믿고 쓸 수 있는가까지 포함한 시스템 설계로 확장됐습니다. 졸업논문으로 단독 수행했고, 전처리부터 모델링, 신뢰도 보정, 해석 가능성, 피드백 계층 분리, UI 구성까지 전체 파이프라인을 직접 만들었습니다.

사용자 불편	핵심 설계	수치 성과
운전자는 모델 수치가 아닌 "지금 무엇을 해야 하는지"를 알고 싶음. 기존 연구는 정확도만 추구 → 확률 과신 시 오경보.	데이터 전처리 → 1D-CNN → Temperature Scaling → Grad-CAM → LLM 피드백. CRITICAL에서는 GPT 배제, 규칙 기반 전환.	Accuracy 0.9765, Macro F1 0.9606. ECE 0.197→0.042 (78.8% 개선). 안전 시스템 책임 경계 설계.

도메인 이해

이상 진단 도메인에서 "잘 맞췄다"와 "믿을 수 있다"는 다른 말입니다.

정확도가 높아도 확률값이 과하게 자신 있어 보이면 오히려 더 위험합니다. 사용자가 그 숫자를 근거로 잘못 안심하거나 반대로 과잉 반응할 수 있습니다. 특히 차량 관련 시스템에서 운전자가 실제로 필요한 건 클래스 분류 결과가 아닙니다. 지금 위험한 상태인지, 당장 뭘 해야 하는지 그게 전부입니다.

생성형 AI를 어디에 둘지도 같은 맥락에서 생각했습니다. LLM은 설명을 풍부하게 만드는 데 강하지만, 책임이 필요한 판단 주체로 두기엔 불확실성이 큽니다. 이 도메인에서는 LLM을 설명 계층으로 제한하는 게 맞다고 봤습니다. 진단을 내리는 역할과 그걸 사람 언어로 풀어주는 역할은 같은 위치에 있으면 안 됩니다.

기존 접근의 한계

기존 이상 진단 연구는 성능 지표 중심으로 평가됩니다. 그 지표들이 필요하다는 건 당연합니다. 다만 실제 사용자 관점에서 빠진 게 있습니다.

예측 확률이 얼마나 믿을 만한지, 모델이 왜 그런 판단을 했는지, 고위험과 저위험 상황을 같은 방식으로 설명할 때 생기는 혼란, 생성형 AI 역할 경계가 흐릴 때 안전 책임도 같이 흐려지는 문제. 이게 논문 수치에는 잘 안 잡히는 부분입니다.

그 중에서 특히 calibration을 중요하게 봤습니다. 정확도가 높아도 확률 보정이 제대로 안 돼 있으면 사용자가 그 숫자를 곧바로 신뢰할 수 없습니다. "얼마나 잘 맞추는가"와 "그 확률을 믿어도 되는가"는 별개의 질문입니다. 둘 다 확인해야 한다고 봤습니다.

설계 관점

세 가지 원칙을 끝까지 유지했습니다.
첫 번째는 모델 성능과 신뢰도를 분리해서 보는 것이었습니다. 분류가 잘 된다고 해서 그 확률이 곧바로 행동 기준이 되지 않습니다. calibration은 부가적인 후처리가 아니라 시스템 신뢰성의 핵심이라고 봤고, 그래서 Temperature Scaling을 통한 보정을 필수 단계로 뒀습니다.

두 번째는 판단 계층과 설명 계층을 분리하는 것이었습니다. WARNING 수준에서는 GPT API를 통해 자연어 피드백을 제공하고, CRITICAL 수준에서는 GPT를 완전히 빼고 규칙 기반 메시지로 강제 전환했습니다. 편의성보다 책임 경계를 먼저 보는 결정이었습니다. 생성형 AI가 어디서든 설명을 잘해준다고 해도, 안전이 걸린 판단 구간에서는 역할을 제한하는 게 맞습니다.

세 번째는 사용자마다 필요한 출력 구조가 다르다는 인식이었습니다. 운전자는 수치보다 행동 지침이 필요하고, 연구자는 모델 근거와 확률 구조를 봐야 합니다. 그래서 하나의 결과를 하나의 화면으로만 처리하지 않고, 운전자 뷰와 연구자 뷰를 분리한 UI를 따로 구성했습니다.

팀 구성과 나의 역할

전처리에서 가장 먼저 붙든 건 leakage 문제였습니다. 10Hz 리샘플링, 윈도우 길이 100 설정, 시간 순서를 유지한 train/val/test 분할을 적용했는데, 데이터를 깔끔하게 정리하는 것보다 실험 구조가 과도하게 낙관적으로 흐르지 않도록 잡는 게 먼저였습니다. 결과가 좋아 보이는 것과 믿을 수 있는 것은 다르고, 그 차이는 전처리 단계에서 이미 결정되는 경우가 많습니다.

1D-CNN 분류 모델은 시계열 신호의 국소 패턴을 반영할 수 있도록 구조를 잡았고, 전체 성능과 클래스별 성능을 함께 확인했습니다. Accuracy 0.9765, Macro F1 0.9606이 나왔습니다.

가장 오래 시간을 쓴 건 Temperature Scaling을 통한 calibration 개선이었습니다. ECE를 0.197에서 0.042로 낮추면서 보정 오류를 78.8% 개선했습니다. Accuracy는 그대로 유지했습니다. 이 작업이 단순 후처리로 보일 수 있는데, 실제로는 사용자가 확률값을 보고 행동을 결정할 수 있게 만드는 단계였습니다. 숫자가 정확한 것과 그 숫자를 믿어도 되는 것은 다릅니다.

Grad-CAM과 채널 중요도 분석으로 해석 가능성을 보강했습니다. 모델이 어떤 신호 구간과 채널을 중요하게 보는지 시각화해서, 결과가 black-box 출력으로 남지 않게 했습니다. 설명을 달아주는 게 아니라 판단 근거를 드러내는 것, 그 차이가 연구자 입장에서는 꽤 중요합니다.

피드백 계층은 WARNING과 CRITICAL을 다르게 처리했습니다. WARNING 수준에서는 GPT API를 통해 자연어로 상황 설명과 행동 권고를 전달하고, CRITICAL 수준에서는 GPT를 완전히 배제하고 규칙 기반 메시지로 강제 전환했습니다. 편의성보다 책임 경계를 먼저 본 결정이었습니다. 운전자 화면에는 보정 확률, 행동 지침, TTS 기반 음성 안내를 뒀고, 연구자 화면에는 혼동행렬, Grad-CAM, 채널 중요도를 따로 배치했습니다. Streamlit 기반 시뮬레이터로 구현해서 실제 사용 시나리오와 분석 시나리오를 동시에 돌려볼 수 있게 만들었습니다.

협업과 개선

처음엔 저도 성능 중심으로 들어갔습니다. 더 좋은 모델 구조, 더 높은 정확도, 더 깔끔한 분류 결과. 그게 자연스러운 출발점이었습니다.

실험을 반복할수록 질문이 달라졌습니다. 클래스를 잘 맞추는 것보다, 이 확률을 운전자가 받았을 때 어떻게 반응할지가 더 자꾸 걸렸습니다. 확률값이 과하게 자신 있어 보이면 오히려 위험합니다. 설명이 없으면 결과를 받아들이기 어렵습니다. CRITICAL 상황에서 GPT가 애매한 답을 내놓으면 책임이 어디에 있는지 불분명해집니다.

그 지점에서 프로젝트 목표를 바꿨습니다. 정확한 모델을 만드는 것에서 믿고 사용할 수 있는 시스템을 만드는 것으로. 이 전환 이후에 calibration, 해석 가능성, 피드백 계층 분리, UI 구조 분리가 전부 핵심 작업이 됐습니다. 모델 파라미터를 다듬는 것보다 이 전환이 훨씬 큰 수정이었습니다.

러닝

ML 시스템은 성능 수치가 높다고 끝나지 않습니다. calibration, explanation, responsibility boundary, 사용자별 출력 구조까지 함께 설계되어야 실제로 쓸 수 있는 시스템이 됩니다. 이 프로젝트 전에는 그게 당연한 말처럼 들렸는데, 직접 만들고 나니 당연하지 않은 일이었습니다.

생성형 AI의 위치에 대한 감각도 여기서 생겼습니다. LLM을 어디에 넣을까보다 어디에는 넣지 말아야 할까를 먼저 보게 됐습니다. 설명을 잘해준다고 해서 판단까지 맡겨도 되는 건 아닙니다. 역할이 명확할 때만 신뢰가 생깁니다.

성장 포인트

앞 네 프로젝트를 돌아보면 하나의 흐름이 있습니다.

조달머신에서는 데이터에 어떤 기준을 부여하느냐가 결과의 가치를 바꾼다는 걸 처음 알았습니다. 선배챗에서는 답변보다 질문 구조가 서비스 품질을 결정한다는 걸 손으로 확인했습니다. 따따에서는 기능보다 사용자 행동이 반복되게 만드는 구조가 먼저라는 걸 개발 현장에서 부딪히며 익혔습니다. 슈토피아에서는 AI를 어떻게 시스템 안에 위치시킬 것인가를 제품 구조 수준에서 처음 고민했습니다.

EV Fault는 그 감각들이 하나로 모인 자리였습니다. 기준 설계, 질문 구조, 사용자 행동 흐름, AI 역할 경계가 전부 이 프로젝트 안에서 동시에 필요했습니다. 졸업논문이라는 형식이었지만 실제로는 지금까지 쌓아온 것들을 한 시스템 안에서 검증하는 작업이었습니다.

그리고 솔직히, 이 프로젝트를 마치고 나서 생긴 게 하나 더 있습니다. 앞으로 어떤 AI 시스템을 만들든 맨 마지막에 스스로에게 던질 질문. 이 결과를 사람이 믿고 행동해도 되는가. 이게 습관처럼 남았습니다. 기술적으로 잘 만든 것과 실제로 써도 되는 것 사이의 거리, 그걸 좁히는 게 결국 이 일의 본질이라고 생각하게 됐습니다.

06 Collaboration — 기준을 만드는 사람, 팀을 함께 만드는 사람

운영 기준이 없을 때, 먼저 구조를 세우는 방식

숭실대 중앙밴드동아리 SOMA 회장 (2020-2021)

코로나 시기에 공연을 운영한다는 건, 기존 방식이 통째로 무력화된 상태에서 처음부터 다시 기준을 잡는 일이었습니다. 관객 수합, 동선, 리스크 대응, 공연 진행 방식까지 전부 새로 정리해야 했고, 참고할 선례도 없었습니다.

그때 회장으로서 가장 먼저 한 건 규칙을 많이 만드는 게 아니었습니다. 구성원들이 각자 다른 판단을 하더라도 같은 방향으로 움직일 수 있는 최소한의 운영 기준을 세우는 것이었습니다. 통제보다 납득이 먼저여야 했습니다. 기준이 납득될 때만 각자의 판단이 조직 전체의 방향과 어긋나지 않는다는 걸 이때 처음으로 직접 겪었습니다.

운영은 이미 있는 프로세스를 반복하는 일이 아니라, 불확실한 상황에서 실행 가능한 기준을 빠르게 정의하는 일이라는 감각이 여기서 생겼습니다. 이후 구름톤에서 멀티 파트를 조율하고, 따따에서 팀 개발 기준을 정리할 때도 그 감각이 그대로 이어졌습니다.

여러 파트가 함께 움직이려면, 공통 언어부터 맞추는 방식

구름톤 유니브 숭실대 부대표, Android 파트장 (2025.03 – 2025.09)

부대표와 Android 파트장을 함께 맡으면서, 개인 작업보다 조직이 반복적으로 굴러갈 수 있는 구조를 정리하는 데 더 많은 시간을 썼습니다.

프론트엔드, 백엔드, 디자인, Android처럼 관심사와 언어가 다른 파트가 함께 움직일 때 일정 관리만으로는 충분하지 않습니다. 각자가 같은 상황을 다르게 해석하면, 조율보다 충돌이 먼저 생깁니다. 그래서 먼저 잡은 건 회칙, 스터디 구조, 파트별 운영 흐름이었습니다. 규칙을 많이 만드는 게 아니라, 구성원들이 반복해서 같은 기준으로 해석할 수 있는 최소한의 공통 구조를 만드는 것이었습니다. 그래야 운영이 특정 사람의 에너지에만 의존하지 않고 조직 차원에서 이어집니다.

Android 파트장으로는 학습과 실습이 분리되지 않도록 스터디 흐름을 설계했고, 부대표로는 파트 간 해석 차이가 운영 충돌로 번지기 전에 기준을 먼저 맞추는 방식으로 조율했습니다. 6개월을 버텨낸 건 사람들이 좋아서이기도 했지만, 초반에 기준을 함께 세웠기 때문이기도 했습니다.

여러 파트가 함께 움직이려면, 공통 언어부터 맞추는 방식

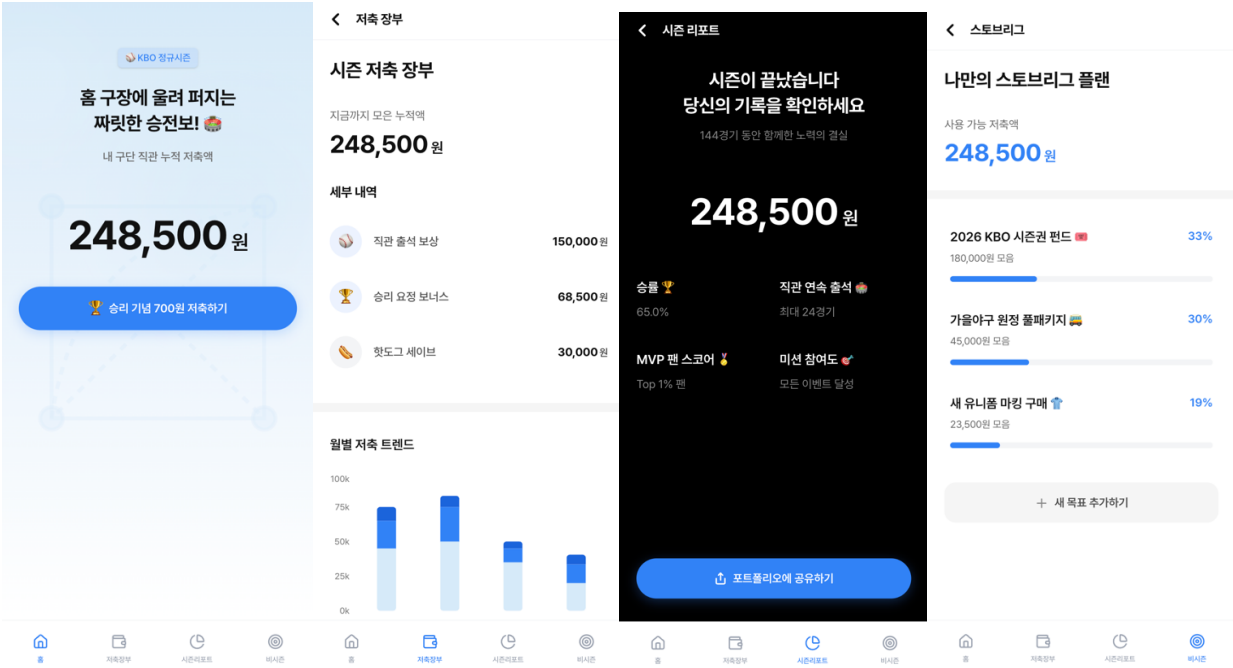
백화점 웰커밍 서비스 그리터 (2021.07 – 현재)

4년 넘게 고객의 첫 접점을 맡으면서 가장 선명하게 남은 건, 같은 말도 상태에 따라 완전히 다르게 들린다는 사실입니다.

불편한 감정을 이미 안고 들어오는 사람, 빠른 안내만 원하는 사람, 설명보다 분위기와 태도로 먼저 신뢰를 판단하는 사람. 이 차이는 정해진 스크립트로 대응하기 어렵습니다. 상황을 먼저 읽고, 이 사람이 지금 무엇을 기대하는지 파악하고, 그에 맞게 응대 방식을 조정하는 것, 그게 실제로 현장에서 반복되는 일의 본질이었습니다.

이 경험이 기술적인 맥락에서도 연결됩니다. 선배챗에서 사용자 상태를 먼저 파악하는 질문 구조를 설계하고, 따따에서 운전자와 연구자에게 다른 출력을 보여주는 UI를 만든 것도, 결국 같은 맥락에서 출발했습니다. 맥락을 먼저 읽어야 그에 맞는 응대가 가능하다는 감각은 현장에서 먼저 생겼고, 프로젝트 안에서 다른 모양으로 이어졌습니다.

07 토스뱅크 AI PM – 내가 기획해보고 싶은 것



금융 개인화의 원료를 바꾸면 어떻게 될까

기존 금융 개인화는 세 가지 데이터로 작동합니다. 소득, 거래 내역, 신용점수. 이 세 가지는 내가 과거에 어떻게 돈을 썼는지를 보여주지만, 지금 이 사람이 어떤 감정 상태인지는 전혀 알려주지 않습니다. 그래서 금융 앱의 개인화는 대부분 "당신의 소비 패턴상 이 상품이 맞습니다" 수준에 머뭅니다.

야구 팬덤 데이터는 다른 종류의 신호입니다. 팬은 144경기 동안 매일 감정 상태가 달라집니다. 연승 구간의 흥분, 연패 속 버팀, 라이벌전 전날 밤의 긴장, 직관 다음 날 아침의 여운. 이 감정 곡선은 금융 행동과 직접 연결됩니다. 기분이 좋을 때 지갑이 열린다는 건 행동경제학의 오래된 관찰이지만, 금융 서비스가 이 신호를 실시간으로 읽고 저축 개입에 쓴 사례는 없습니다.

이 기획의 출발점은 여기입니다. 팬의 감정 리듬을 금융 개인화의 원료로 쓰면 어떻게 될까.

기존 서비스가 못 하는 것

신한은행 KBO 적금은 팬 데이터를 쓰지 않습니다. 응원팀 선택과 시즌 종료 성적 정도가 전부입니다. 팬이 144경기 동안 만들어 낸 행동 데이터인 몇 번 직관을 갔는지, 연패 속에서도 체크인을 했는지, 어떤 경기 유형에서 반응이 컸는지는 금리 계산에 들어가지 않습니다.

더 근본적으로, 기존 적금은 저축 금액과 목표가 가입 시점에 고정됩니다. 팀이 선두권으로 치고 올라가도, 연패로 흥미를 잃어가도, 한국시리즈 직관이 갑자기 간절해져도 적금은 그 변화를 읽지 못합니다. 이 제품이 풀려는 문제는 명확합니다. 팬의 감정 상태와 행동 패턴을 실시간으로 읽어 저축 금액, 타이밍, 목표를 동적으로 최적화하는 것입니다.

AI/ML 설계 — 왜 이 기술이 이 자리에 있어야 하는가

이 제품에서 AI가 필요한 이유는 하나입니다. 규칙 기반으로는 "이 사람에게 지금 얼마를, 언제 제안해야 하는가"를 풀 수 없기 때문입니다.
팬마다 저축 행동이 일어나는 트리거가 다릅니다. 승리 직후 흥분 상태에서 바로 반응하는 팬이 있고, 경기 다음 날 아침 여운이 남아 있을 때 반응하는 팬이 있습니다. 어떤 팬은 연승 구간에서, 어떤 팬은 직관 인증 직후에 저축 실행률이 높습니다. 이 패턴은 고정된 규칙으로 만들 수 없고, 개인별로 학습해야 합니다.

입력 신호도 기존 금융 데이터와 다릅니다. 시즌 컨텍스트(현재 순위, 연승/연패 구간, 경기 중요도), 팬 행동(체크인 지속률, 직관 인증 빈도, 앱 방문 패턴), 금융 행동(최근 적립 실행률, 목표 대비 페이스)이 함께 작동해야 의미 있는 추천이 나옵니다. 어떤 신호를 피처로 쓸지를 정의하는 것 자체가 이 제품의 핵심 기획 과제입니다.

추천 방향은 탐색과 활용을 동시에 해야 하는 문제 특성상 Bandit 계열 접근이 자연스럽다고 봅니다. 어떤 금액 제안이 이 사람의 저축 실행으로 이어지는지 초기에는 모르기 때문입니다. 다만 정확한 모델 설계는 ML Engineer와 함께 데이터를 보면서 결정해야 할 영역입니다. 초기에는 규칙 기반으로 시작해 데이터가 쌓이면서 점진적으로 고도화하는 방향이 현실적입니다.

이탈 예측도 필요합니다. 이탈은 갑자기 일어나지 않습니다. 앱 방문 감소, 팬 행동 참여 저하, 팀 성적 악화가 며칠에 걸쳐 누적됩니다. 이 신호들이 겹치는 시점을 포착해 개입 타이밍을 만드는 것이 리텐션의 핵심입니다. 초기에는 간단한 규칙 기반 스코어링으로 시작하고 이후 모델로 고도화하는 순서가 맞습니다.LLM은 판단이 아니라 전달에만 씁니다. 저축 금액 계산과 타이밍 판단은 모델이 합니다. LLM은 그 결과를 팬의 언어로 번역하는 마지막 단계에서만 작동합니다. 오늘의 홈 화면 카피, 시즌 리포트 문장, 오프시즌 목표 제안 코멘트가 여기에 해당합니다. 판단 책임이 있는 구간에 LLM을 두지 않는 것은, 신뢰도가 중요한 금융 서비스에서 특히 중요한 원칙입니다..

제품이 만드는 경험

홈 화면은 오늘의 감정 상태를 읽은 문장으로 시작합니다. 승리한 날 "오늘 이겼습니다. 이 기세, 기록해볼까요?"와 연패 중인 날 "힘든 시즌이지만 당신의 응원은 멈추지 않았습니다. 오늘도 200원만요."는 같은 화면이지만 다르게 읽힙니다. 저축 금액 제안도 감정 상태를 반영해 달라집니다.

통장 화면은 기본 적립, 승리 보너스, 팬 행동 보상이 분리된 구조로 보입니다. 팀 성적이 나빠도 내 통장은 다른 이유로 계속 쌓입니다. 연패 속에서도 끝까지 응원한 팬이 승리만 챙긴 팬보다 시즌 총 적립액이 더 클 수 있는 구조입니다.

시즌 종료 시점에는 리포트가 생성됩니다. "뜨거웠던 144경기, 당신의 기록 — 248,500원." 그 아래 LLM이 만든 문장. "이번 시즌 우리 팀은 76승 68패를 기록했고, 당신은 연패 구간에서도 112경기를 함께했습니다. 승리뿐 아니라 버텼던 시간까지 기록으로 남았네요." 이 리포트는 포토 카드로 저장되고 공유됩니다. 금융 앱에서 사람들이 자발적으로 공유하는 콘텐츠는 없습니다. 이게 그 예외가 될 수 있습니다.

오프시즌 모드에서는 지난 시즌 행동 데이터 기반으로 목표를 추천합니다. 직관을 자주 다녀온 팬에게는 내년 시즌권 자금을, 굿즈 소비가 많았던 팬에게는 유니폼 장만 목표를, 원정 체크인이 많았던 팬에게는 원정 응원 투어 자금을 제안합니다. 이 사람이 실제로 어떤 팬인지를 읽은 추천입니다.

성공 지표와 실험 구조

북극성 지표는 시즌 중반 이후 적립 유지율입니다. 60경기 이후에도 계속 저축하는 사람의 비율이 이 제품의 실효성을 보여줍니다.
첫 번째 실험은 개인화 추천 효과입니다. 고정 금액 추천 그룹과 개인화 추천 그룹의 4주 납입 지속률과 저축 실행률을 비교합니다.
두 번째 실험은 타이밍 최적화입니다. 팬별 최적화된 푸시 타이밍과 고정 타이밍의 실행률 차이를 측정합니다.
세 번째는 하위권 팬 리텐션 보정입니다. 팬 행동 보상 강화 정책이 연패 구간에서 이탈률을 얼마나 줄이는지, 강팀 팬과 약팀 팬의 리텐션 편차를 별도 지표로 관리합니다.

기술보다 먼저 정의해야 할 것들

ML Engineer, Data Scientist와의 협업 구조도 이 제품 안에서 자연스럽게 생깁니다. 어떤 팬 행동을 금융 신호로 볼 것인지는 데이터 과학의 문제이기 전에 제품 철학의 문제입니다. 그 정의를 PM이 먼저 잡고 들어가야 모델 설계가 방향을 찾습니다.

연패가 열흘째 이어지는 8월, 비가 오는 날에도 직관을 다녀온 팬. 그 행동이 금융 서비스 안에서 읽히고 기록될 수 있어야 한다는 생각에서 이 기획이 시작됐습니다. 그걸 실제로 만들어보는 일을 토스뱅크에서 해보고 싶습니다.